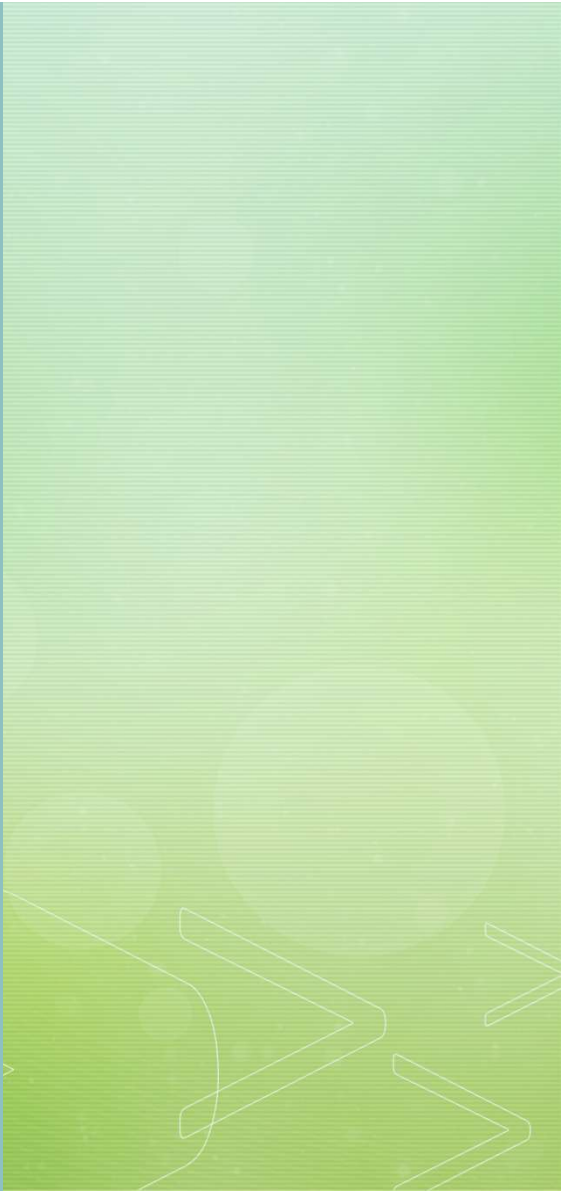




Plots in R



R itself comes pre-loaded with basic R graphics functions:

Base R Function	Type of Graph	Graph Description
<code>barplot()</code>	bar graph	categorical data with bars with heights/lengths proportional to the values
<code>boxplot()</code>	box-and-whisker plot	shows distribution of data points through quartiles
<code>dotchart()</code>	Cleveland dotplot	an alternative to a bar graph; plots a dot for each observation on a scale
<code>hist()</code>	histogram	distribution of numerical data
<code>plot()</code>	scatterplot	plotted dots represent values for two numeric variables on the horizontal and vertical axes
<code>pie()</code>	piechart	displays data as a percentage of a whole



Important packages

- graphics library (automatically loads when we load R)
- grid library
- grDevices library
- ggplot2 library
- plot3D for multi-dimensional plots

Redirecting output to a file

Function	Comments
<code>svg("mygraph.svg")</code>	Recommended: Resize without fuzziness or pixelation
<code>pdf("mygraph.pdf")</code>	Recommended: Resize without fuzziness or pixelation
<code>win.metafile("mygraph.wmf")</code>	Ideal for graphs used in Microsoft Word or PowerPoint
<code>png("mygraph.png")</code>	Recommended for webpages
<code>postscript("mygraph.eps")</code>	Suitable for embedding an image in documents, or sending to printers
<code>tiff("mygraph.tiff")</code>	Large file size
<code>jpeg("mygraph.jpg")</code>	Generally for photographs/pixelated images
<code>bmp("mygraph.bmp")</code>	Seldom used outside of Windows operating system

plot() function

- `xval = seq(1,10,0.5)`
- `yval = 30*xval/(2+xval)`
- `plot(xval,yval)`

plot() option: Point character options

pch=0	square
pch=1	circle
pch=2	triangle point up
pch=3	plus
pch=4	cross
pch=5	diamond
pch=6	triangle point down
pch=7	square cross
pch=8	star
pch=9	diamond plus
pch=10	circle plus
pch=11	triangles up and down
pch=12	square plus
pch=13	circle cross
pch=14	square and triangle down
pch=15	filled square blue
pch=16	filled circle blue
pch=17	filled triangle point up blue
pch=18	filled diamond blue
pch=19	solid circle blue
pch=20	bullet (smaller circle)
pch=21	filled circle red
pch=22	filled square red
pch=23	filled diamond red
pch=24	filled triangle point up red
pch=25	filled triangle point down red

plot() option: Size of the data point

The parameter for changing the size of the data point is `cex`. This parameter takes values in units of 'fraction of default pixel size of fonts set in the machine and thus acts as a scaling parameter for the fonts.

`cex` ---> A number indicating the amount by which plotting text and symbol should be scaled relative to the default value.

Thus, `cex = 1` is default size
`cex = 1.5` is 150% of default size
`cex = 0.5` is 50% of default size

[Note : `cex.axis` --> scales the axis
`cex.lab` ---> scales the label
`cex.main` --> scales main title
`cex.sub` ---> scales the subtitle]

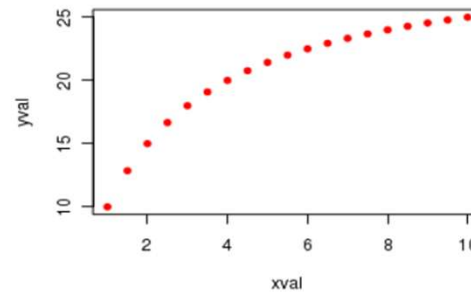
Plot() option: Color of the data point

- Colors()
- col="blue", col="red"
- Col can also be hexadecimal values

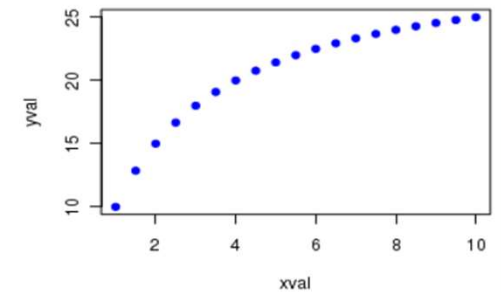
```
col = "#A9F3BB"  -----> colour corresponding to Red=A9, Green=F3, Blue=BB in  
                    hexadecimal representation.  
  
col = "#FF0000"  -----> colour corresponding to Red=FF, Green=00, Blue=00 in  
                    hexadecimal representation.  
                    ( This is maximum strong red colour, since  
                      Blue is 00, and green is 00. ).
```

Examples

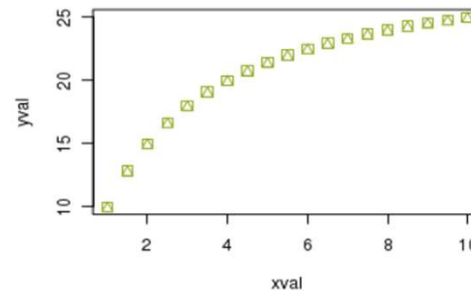
- `plot(x,y,pch=20,cex=1,col="red")`
- `pch=20, cex=1.2, col="blue"`
- `pch=14, cex=1.4, col="#99AA28"`
- `pch=18, cex=1.8, col="#33AAF8"`



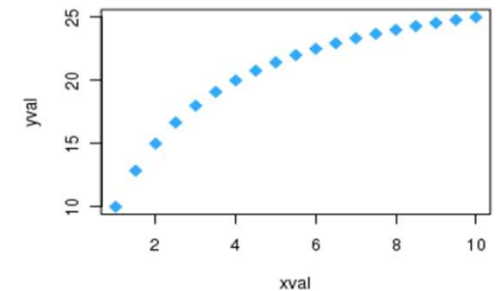
`pch=20, cex=1.0, col="red"`



`pch=20, cex=1.2, col="blue"`

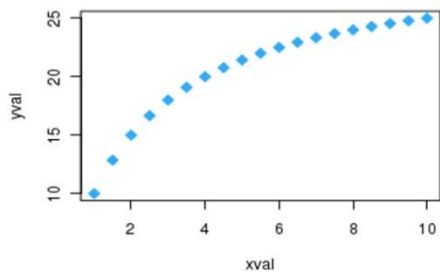


`pch=14, cex=1.4, col="#99AA28"`

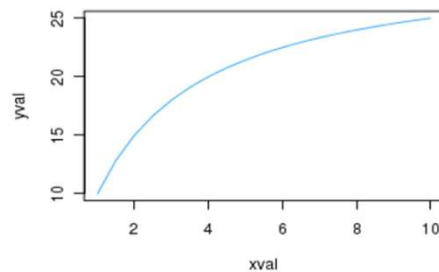


`pch=18, cex=1.8, col="#33AAF8"`

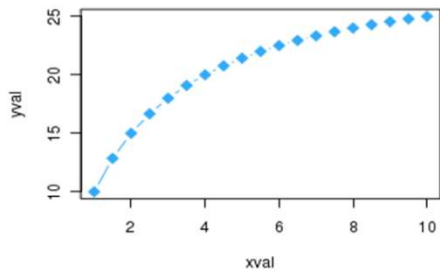
plot() option: type parameter for joining data points



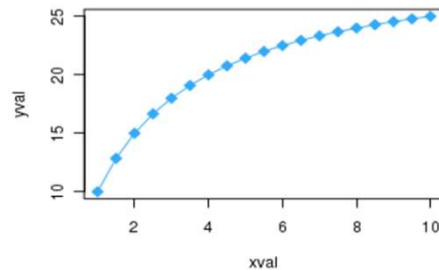
type="p" (Only points)



type="l" (Only line)

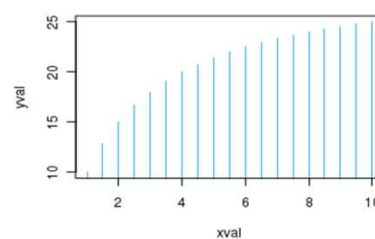


type="b" (line joining points)

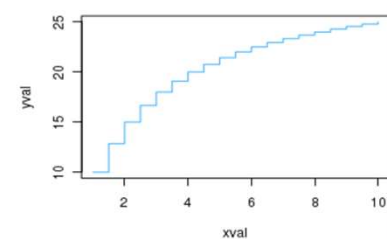


type="o" (line overlaying points)

```
type="p"  plots points
type="l"  plots lines
type="b"  plots points and lines
type="o"  plots points overlaid by lines
type="h"  plot with histogram like vertical lines
type="h"  plot with histogram like vertical lines
type="s"  plot with stair steps
type="n"  no plotting - blank plot with axis marked (x,y)
```



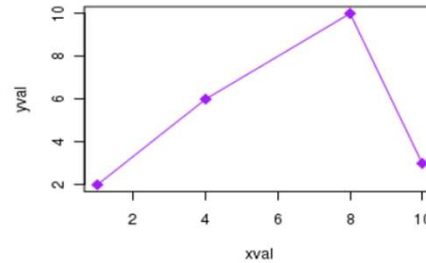
type="h" (histogram like)



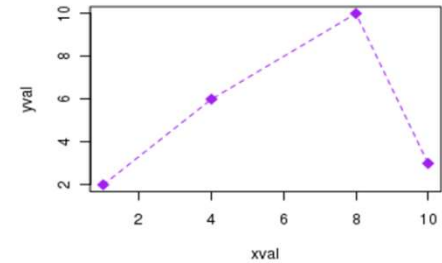
type="s" (steps)

Linetype (lty) and linewidth (lwd) option for plot()

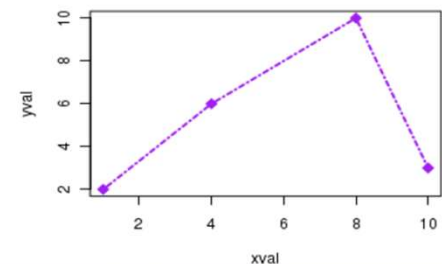
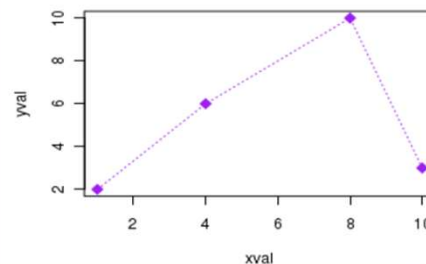
<code>lty = 0</code>	or	<code>lty = "blank"</code>
<code>lty = 1</code>	or	<code>lty = "solid"</code>
<code>lty = 2</code>	or	<code>lty = "dashed"</code>
<code>lty = 3</code>	or	<code>lty = "dotted"</code>
<code>lty = 4</code>	or	<code>lty = "dotdash"</code>
<code>lty = 5</code>	or	<code>lty = "longdash"</code>
<code>lty = 6</code>	or	<code>lty = "twodash"</code>



`lty=1, lwd=1`



`lty=2, lwd=1`



Plot title and its properties

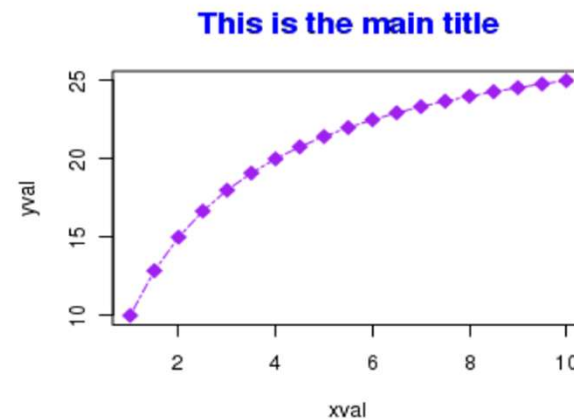
`col.main` ---> sets the color of main title
Takes same values as 'col'

`font.main` ---> sets the font of main title.
`font.main = 1` for plain
`font.main = 2` for bold
`font.main = 3` for italic
`font.main = 4` for bold italic

`cex.main` ---> scales the main title, as explained before.

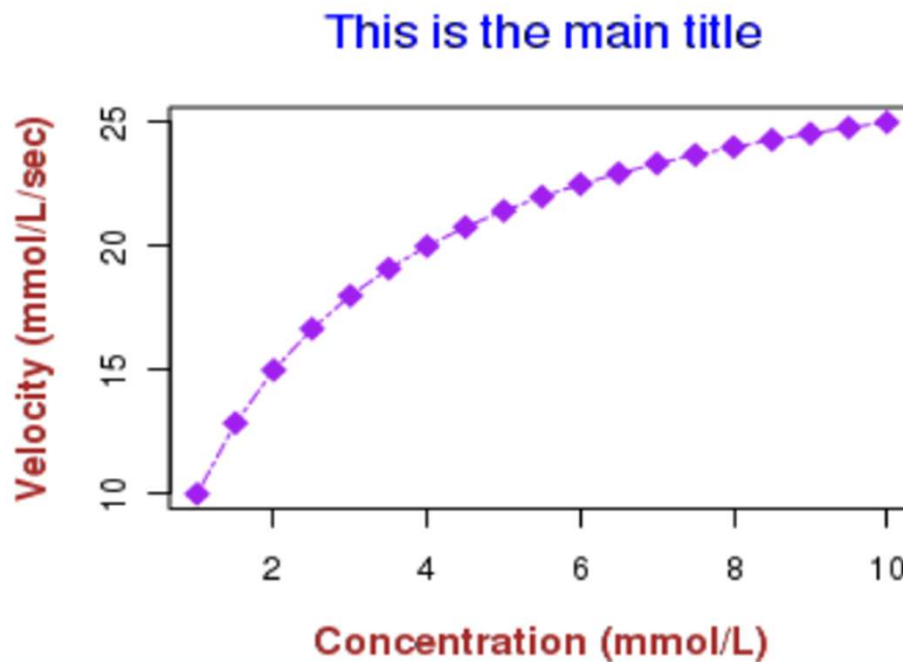
```
plot(xval, yval, pch=18, cex=1.8, col="purple", type="o", lty=6, lwd=1,  
     main="This is the main title", col.main="blue", font.main=2, cex.main=1.5 )
```

The above code creates the following plot with main title:



Axes titles and their properties

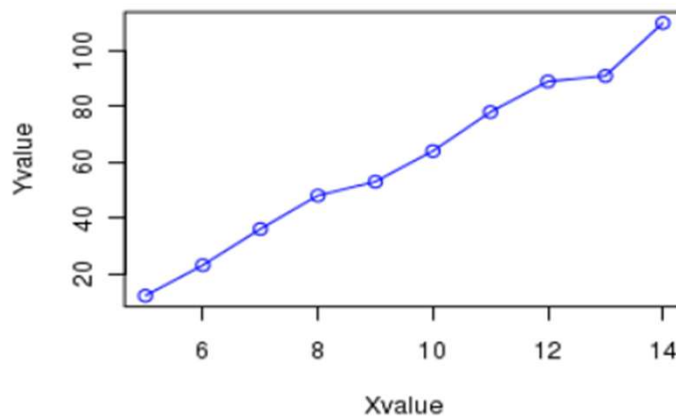
```
plot(xval, yval, pch=18, cex=1.8, col="purple", type="o", lty=6, lwd=1,
     main="This is the main title", col.main="blue", font.main=1, cex.main=1.5,
     xlab = "Concentration (mmol/L)", ylab = "Velocity (mmol/L/sec)",
     font.lab = 2, col.lab="brown", cex.lab = 1.2)
```



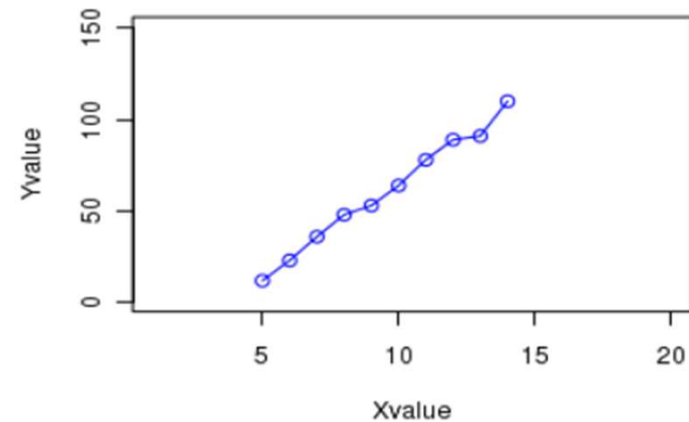
Font,
Color,
Size

Axes ranges

```
# Range is set using xlim and ylim parameters  
plot(Xvalue, Yvalue, col="blue", type="o", xlim=c(1,20), ylim=c(1,150))
```



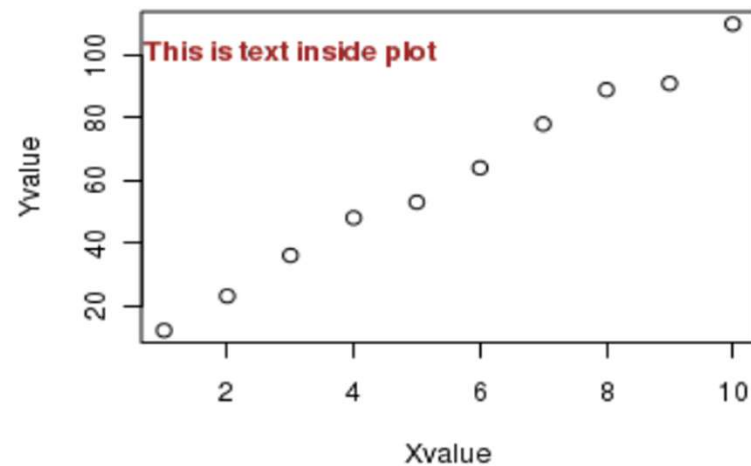
Default range selection



xlim=c(1,20), ylim=c(1,150)

Text inside a plot

```
# We add text whose center is at a particular location (3, 105) in the plot.  
# First look at the plot, and decide the units for (x,y)!!  
plot(Xvalue, Yvalue, text(3,100,"This is text inside plot", col="brown", font=2))
```



Text centered at (3,100) inside the figure

Text next to data points

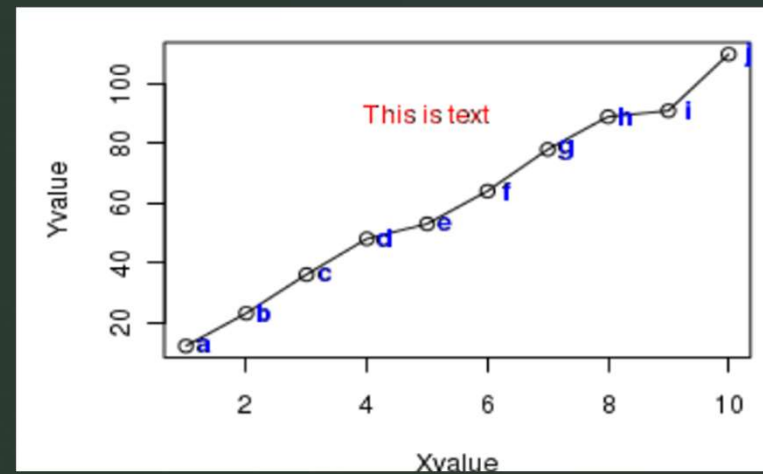
```
Xvalue = c(1,2,3,4,5,6,7,8,9,10)
Yvalue = c(12, 23, 36, 48, 53, 64, 78, 89, 91, 110)

# these letters are labels for data points.
cch = c("a","b","c","d","e","f","g","h","i","j")

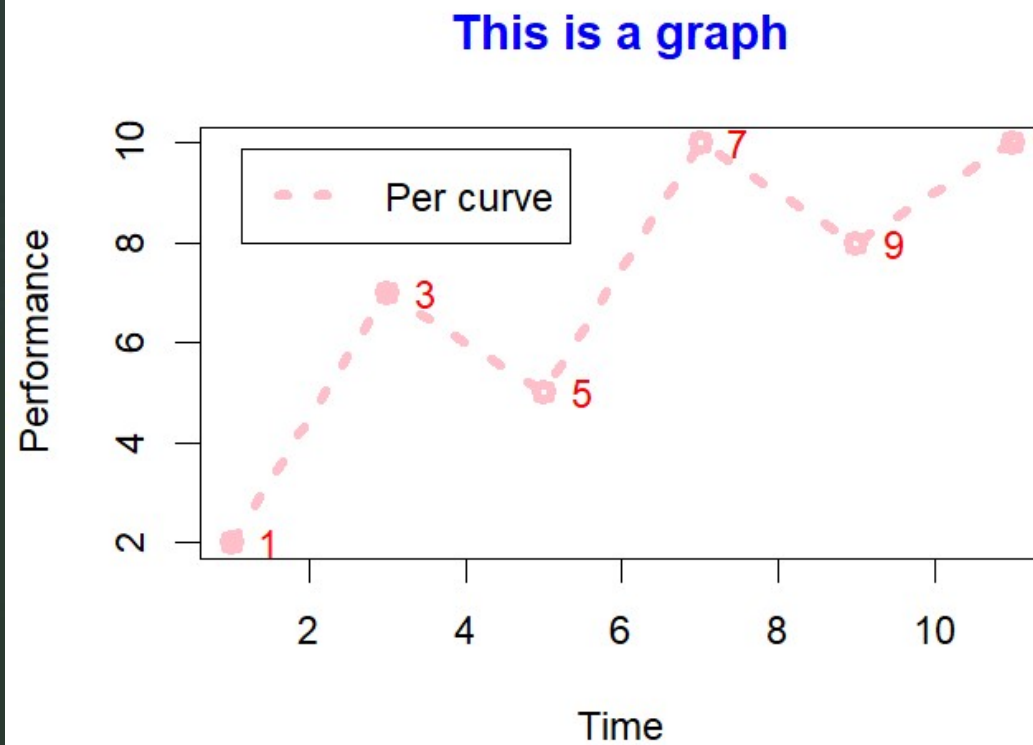
# plotting the graph
plot(Xvalue,Yvalue, type="o")

# adding text label to each point
text(Xvalue+0.3, Yvalue, cch, col="blue", font=2)

# adding a second text string inside the plot.
text(5,90, "This is text", col="red")
```

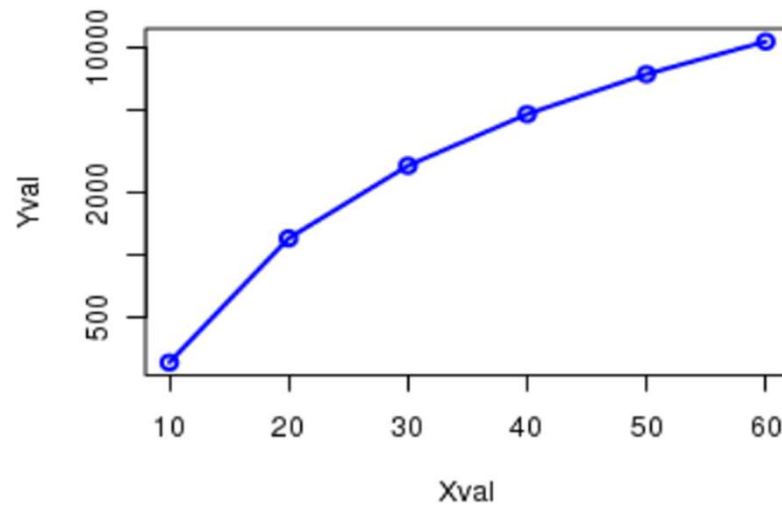


Another example



Logarithmic scales

```
plot(Xval, Yval, log="y", type="o", col="blue", lwd=2)
```



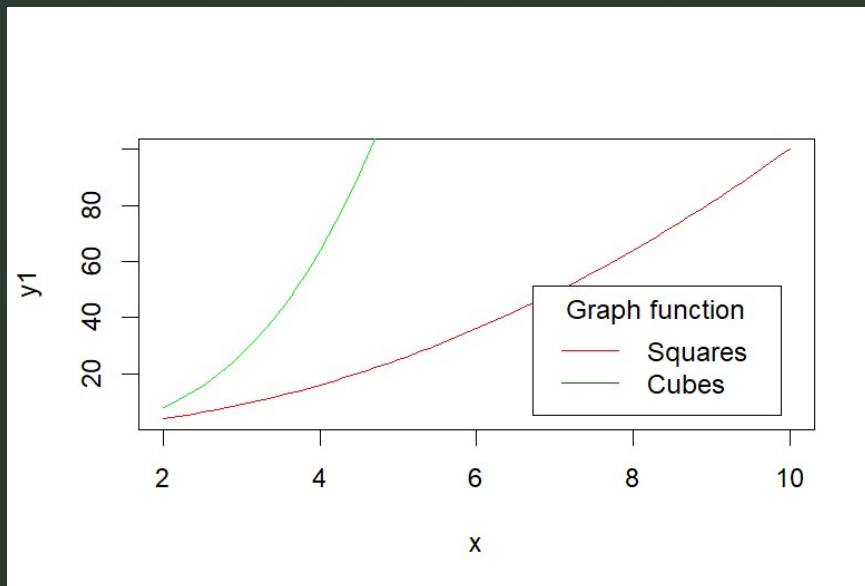
Logarithmic scale on the Y axis

Multiple graphs on the same plot

- `par(mfrow=c(1,2))` # 1 row and 2 columns
- `x=seq(-pi,pi,0.1)`
- `y=sin(x)`
- `plot(x,y,type='l')`
- `plot(x,y,pch=c(4,5,6),color=('red','blue','violet','green'))`

Overlaying graphs

```
x <- seq(2,10,0.1)
y1=x^2
y2=x^3
plot(x,y1,type='l',col='red')
lines(x,y2,col='green')
legend('bottomright',inset=0.05,c("Squares", 'Cubes'),lty=1,col=c('red', 'dark green'),title="Graph function")
```



Adding lines to the graph

```
x <- seq(2,10,0.1)
y1=x^2
y2=x^3
plot(x,y1,type='l',col='red')
lines(x,y2,col='green')
legend('bottomright',inset=0.05,c("Squares", 'Cubes'),lty=1,col=c('red', 'dark green'),title="Graph function")
```

```
abline(a=4,b=5,col='blue')
abline(h=c(4,6,8),col="dark green",lty=2)
abline(v=c(4,6,8),col="dark green",lty=1)
```

